PRACTICAL - 4

AIM - Find the Maximum Temperature Recorded for Each Year of NCDC Data using MapReduce

Problem Definition

A meteorological research organization maintains historical weather records collected from various weather stations worldwide. The organization uses Hadoop to analyze large climate datasets and study long-term temperature trends. As a Data Analytics Engineer, you are required to process the National Climatic Data Center (NCDC) dataset and identify the maximum temperature recorded for each year using MapReduce.

Task 1: Upload the NCDC weather dataset to HDFS and examine the structure of the weather records.
Task 2: Develop and implement the Mapper program to extract the year and temperature values from each record and generate intermediate key-value pairs.
Task 3: Develop and implement the Reducer program to determine the maximum temperature
recorded for each year.
Task 4: Execute the MapReduce job on Hadoop and store the results in HDFS.
Verify the correctness of the output generated.
Task 5: Analyze the yearly maximum temperature trends and identify the hottest year in the dataset. Discuss the advantages of using MapReduce for large-scale climate data analysis.

```
@24dit026-tech →/workspaces/BDA-SHP-2026/Practical-4 (main) $ cat > ncdc_data.txt <<'EOF'
1901,32
1901,45
1901,28
1901,51 …
1904,49
1904,67
1905,44
1905,58
1905,71
EOF
@24dit026-tech →/workspaces/BDA-SHP-2026/Practical-4 (main) $ cat ncdc_data.txt
1901,32
1901,45
1901,28
1901,51
1902,39
1902,48
1902,55
1902,41
1903,35
1903,62
1903,47
1904,52
1904,49
1904,67
1905,44
1905,58
1905,71
@24dit026-tech →/workspaces/BDA-SHP-2026/Practical-4 (main) $
```

CREATING NCDC DATASET

```
@24dit026-tech →/workspaces/BDA-SHP-2026/Practical-4 (main) $ cat > mapper.py <<'EOF'
import sys

for line in sys.stdin:
    line = line.strip()

    if not line:
        continue

    year, temperature = line.split(",")

    print(f"{year}\t{temperature}")
EOF
@24dit026-tech →/workspaces/BDA-SHP-2026/Practical-4 (main) $ cat mapper.py
import sys

for line in sys.stdin:
    line = line.strip()

    if not line:
        continue

    year, temperature = line.split(",")

    print(f"{year}\t{temperature}")
```

MAPPER CODE

```
@24dit026-tech →/workspaces/BDA-SHP-2026/Practical-4 (main) $ python3 mapper.py < ncdc_data.txt
1901    32
1901    45
1901    28
1901    51
1902    39
1902    48
1902    55
1902    41
1903    35
1903    62
1903    47
1904    52
1904    49
1904    67
1905    44
1905    58
1905    71
```

MAPPER_OUTPUT

The Mapper reads each weather record, extracts the year and temperature, and generates an intermediate key-value pair in the form (year, temperature)

```
@24dit026-tech →/workspaces/BDA-SHP-2026/Practical-4 (main) $ cat reducer.py
import sys

current_year = None
maximum_temperature = None

for line in sys.stdin:
    line = line.strip()

    if not line:
        continue

    year, temperature = line.split("\t")
    temperature = int(temperature)

    if current_year == year:
        maximum_temperature = max(maximum_temperature, temperature)
    else:
        if current_year is not None:
            print(f"{current_year}\t{maximum_temperature}")

        current_year = year
        maximum_temperature = temperature

if current_year is not None:
    print(f"{current_year}\t{maximum_temperature}")
@24dit026-tech →/workspaces/BDA-SHP-2026/Practical-4 (main) $
```

REDUCER

The Reducer expects grouped/sorted Mapper output

```
@24dit026-tech →/workspaces/BDA-SHP-2026/Practical-4 (main) $ python3 mapper.py < ncdc_data.txt | sort -k1,1 | pyth
1901    51
1902    55
1903    62
1904    67
1905    71
@24dit026-tech →/workspaces/BDA-SHP-2026/Practical-4 (main) $
```

MAX_TEMP_OUTPUT

```
@24dit026-tech →/workspaces/BDA-SHP-2026/Practical-4 (main) $ python3 mapper.py < ncdc_data.txt | sort -k1,1
ort -k2,2nr | head -1
1905    71
@24dit026-tech →/workspaces/BDA-SHP-2026/Practical-4 (main) $
```

HOTTEST_YEAR

1905 was the hottest year with a maximum temperature of 71

Key Questions / Analysis

Q1. How is weather data parsed and processed using MapReduce?

The weather dataset contains records with a year and temperature value. The Mapper reads each record and separates the year and temperature. It generates an intermediate key-value pair containing the year and its temperature. Hadoop then groups the records belonging to the same year and sends them to the Reducer. The Reducer processes these temperature values and finds the maximum temperature for each year.

Q2. What key-value pairs are generated by the Mapper?

The Mapper extracts the year as the key and the temperature as the value. For example, if the input record is 1901,45, the Mapper generates (1901, 45). Similarly, for 1901,51, it generates (1901, 51). These intermediate pairs are then grouped according to the year before being passed to the Reducer.

Q3. How does the Reducer identify the maximum temperature for a given year?

The Reducer receives all temperature values belonging to the same year. It compares these values one by one and keeps the highest value. For example, if the values for 1901

are 32, 45, 28 and 51, the Reducer compares them and produces 1901, 51. In this way, the maximum temperature is calculated separately for every year.

Q4. What challenges arise when processing large climate datasets?

Large climate datasets can contain millions of records collected from many weather stations. Processing such a large amount of data on a single computer can require a lot of memory, storage and processing time. The data may also have missing, incorrect or differently formatted records. MapReduce helps handle these challenges by dividing the data and processing it in smaller parts across multiple machines.

Q5. How does Hadoop improve scalability and performance for weather data analytics?

Hadoop improves scalability by distributing large datasets and processing tasks across multiple machines. MapReduce allows different parts of the weather data to be processed in parallel, reducing the workload on a single machine. HDFS provides distributed storage, while MapReduce performs the computation. Therefore, Hadoop can efficiently process much larger climate datasets than a traditional single-machine approach.

Dataset / Test Data

NCDC Weather Dataset — Historical Temperature Records; sample climate data files stored in HDFS